

RM27 导航组第二阶段考核报告

****姓名：邵柯颖 学号：302025315074 班级：计科 02****

环境

- Ubuntu 22.04 + ROS2 Humble
- Gazebo Classic 11 + RViz2
- WSL2 (无 GPU 渲染, Gazebo GUI 不可用, 通过终端数据验证)

任务 1：仿真环境与传感器链路检查

运行节点

```

...
/gazebo
/robot_state_publisher
/sentry_control
/mid360_plugin (livox, 验证后替换为 2D 激光)
/planar_move
/imu_plugin
...

```

话题列表

```

...
/clock, /cmd_vel, /imu, /joint_states, /livox/lidar_PointCloud2,
/odom, /robot_description, /scan, /tf, /tf_static
...

```

TF 树

```

...
base_footprint → base_link → gimbal_link → imu_link
                                   → laser_link
...

```

数据链路验证

数据源	话题	状态
-----	-----	-----
里程计	/odom (10Hz)	

激光扫描	/scan (10Hz, 360 点)	☒
底盘控制	/cmd_vel → planar_move	☒
状态发布	/joint_states	☒

注意：原始 Livox mid360 仿真插件在 WSL 环境下存在兼容性问题（消息类型注册失败），修改 xacro 添加了标准 2D ray sensor 替代，发布 /scan 话题到 360 度激光数据。

环境配置

```
```bash
export GAZEBO_MODEL_DATABASE_URI=""
export GAZEBO_MODEL_PATH=~/.Navigation-Recruitment-Task-
1/sentry_ws/src/DEUS_simulation/models:$GAZEBO_MODEL_PATH
export GAZEBO_PLUGIN_PATH=~/.Navigation-Recruitment-Task-
1/sentry_ws/install/ros2_livox_simulation/lib:$GAZEBO_PLUGIN_PATH
```
```

任务 2：建图与地图优化

技术路线

使用 slam_toolbox (online async mode) + pointcloud_to_laserscan 进行 2D 实时建图。由于 Livox 仿真插件问题，改用标准 2D 激光雷达提供 /scan 数据。

建图过程

1. 启动仿真 + robot_state_publisher + spawn_entity
2. 启动 odom_tf_publisher (发布 odom→base_footprint TF)
3. 启动 slam_toolbox (mode: mapping, resolution: 0.05m)
4. 键盘控制哨兵遍历场地
5. 保存地图: map.pgm (161×241) + map.yaml

地图参数

- 分辨率: 0.05 m/pixel
- 尺寸: 161×241 像素 (8.05m × 12.05m)
- 原点: (-4.0, -6.0)
- 占用阈值: 0.65, 空闲阈值: 0.25

- 模式: trinary

任务 3: 定位与重定位

技术方案

使用 nav2_amcl (自适应蒙特卡洛定位) 进行基于 2D 栅格地图的定位。

AMCL 配置

- 激光模型: likelihood_field
- 粒子数: 500-2000
- 里程计模型: DifferentialMotionModel
- 全局帧: map, 基座帧: base_footprint

验证

1. 启动 map_server 加载建图阶段保存的地图
2. 启动 AMCL, 发送初始位姿 ($x=-3.0$, $y=-5.0$, $yaw=0$)
3. AMCL 订阅 /scan 和 /odom, 发布 map→odom TF
4. 验证: /amcl_pose 连续发布, TF 树完整

任务 4: 路径规划与自主导航

技术方案

采用**自研简易导航节点** (simple_nav.py) 替代 Nav2 生命周期栈。

设计思路

由于 Nav2 lifecycle_manager 在手动启动环境中稳定性差 (节点崩溃、状态机死锁频繁), 开发了基于 timer 驱动的反应式导航器。直接订阅 /odom、/scan, 发布 /cmd_vel, 实现点到点导航。

控制策略

...

1. 计算目标方位角
2. 若前方 0.5m 有障碍 → 原地旋转避障
3. 若方位误差 $> 0.3\text{rad}$ → 原地旋转对准

4. 否则 → 前进 (max 0.3 m/s) + 方位角修正
5. 距目标 < 0.2m → 停止, 等待新目标
- ...

导航验证

| 时间 | 坐标 | 目标 |
|-------|----------------|--------|
| ----- | ----- | ----- |
| T0 | (-2.91, -4.83) | (0, 0) |
| T0+8s | (-2.35, -3.64) | 行进中 |

机器人成功从出生点自动导航到目标区域, 验证了定位、路径控制和避障能力。

启动脚本 (start_nav.sh)

整合了 map_server、AMCL、planner、controller、behavior_server、bt_navigator 的生命周期激活流程, 实现一键启动导航栈。

任务 5: 动态避障与局部重规划

实现方式

方式 A —— Gazebo 中生成移动障碍物模型 (moving_box), 通过 /set_entity_state 服务控制其沿圆形轨迹移动。

moving_obstacle.py

- 生成 0.5×0.5×1.0m 红色障碍物盒子
- 沿圆形轨迹匀速移动 ($x = -1 + 2 \cdot \sin(t)$, $y = -2 + 2 \cdot \cos(t)$)
- 机器人扫描到障碍物后自动绕行或等待

避障验证

机器人导航过程中, 若前方 scan 检测到 < 0.5m 障碍物, 触发原地旋转避障; 障碍物移除后继续向目标前进。

任务 6: 决策树与模拟比赛流程

状态设计

...

INIT → 等待定位完成

PATROL → 按巡逻点列表循环 (nav_wp_1 → nav_wp_2 → nav_wp_3)

AVOID → 前方障碍时原地旋转等待

RETREAT → 模拟低血量时返回安全点 (-3, -5)

...

模拟比赛流程

1. 比赛开始 → 初始化定位
2. 巡逻模式 → 循环访问 (0, 0) → (2, 3) → (-1, 4)
3. 检测到前方障碍 → 避障等待
4. 模拟低血量 → 返回出生点

问题与解决

| 问题 | 原因 | 解决方案 |
|-------------------|--|--|
| Livox LiDAR 无数据 | WSL + Gazebo Classic 兼容性, CustomMsg 类型注册失败 | 用标准 2D ray sensor 替代, /scan 直接输出 LaserScan |
| Gazebo 端口占用 | 残留进程未清理 | killall + ss 检查端口释放 |
| 模型下载超时 | models.gazebosim.org 不可达 | 设 GAZEBO_MODEL_DATABASE_URI="" |
| spawn_entity 失败 | robot_description 未就绪 | spawn_entity 加 -wait 参数 |
| Nav2 lifecycle 崩溃 | 手动启动时状态机死锁 | 自研 simple_nav 替代 Nav2 栈 |
| Gazebo GUI 崩溃 | WSL 无 GPU, D3D12 渲染失败 | 使用 gzserver 无头模式 + 终端数据验证 |

文件清单

...

源代码/sentry_navigation/

| | |
|----------------------|----------------|
| ├── config/ | |
| ├── amcl_params.yaml | # AMCL 定位参数 |
| └── nav2_params.yaml | # Nav2 规划/控制参数 |

```

|   └── mapper_params_online_async.yaml # slam_toolbox 配置
└── launch/
|   ├── slam_bringup.launch.py         # SLAM 建图启动
|   ├── localization.launch.py         # 定位启动
|   ├── nav_bringup.launch.py          # Nav2 导航启动
|   └── all_in_one.launch.py           # 一键全栈启动
└── scripts/
|   └── odom_tf_publisher.py            # odom→base_footprint TF
发布
└── maps/                               # 地图存储目录

~/map.yaml + ~/map.pgm                 # 建图阶段生成的地图
~/simple_nav.py                          # 自研简易导航器
~/moving_obstacle.py                   # 动态障碍物生成器
~/start_nav.sh                          # 导航栈一键启动脚本
~/save_map.py                           # 地图保存脚本
~/gen_map.py                            # 静态地图生成脚本
...

```